Essential Git Commands Cheat Sheet

A Practical Guide for Version Control and Collaborative Development

1. Repository Initialization & Setup

These commands are used to start a new repository, download existing codebases, and configure your local identity for version tracking.

Command	Description & Notes
`git init`	Initializes a brand new, empty Git repository in your current working directory.
`git clone <url>`	Downloads an existing remote repository and its complete version history onto your local machine.
`git config --global user.name "Name"`	Sets your global author name attached to every commit you make.
`git config --global user.email "email"`	Sets your global email address linked to your version history.

2. Staging and Committing Changes

Version control relies on staging modifications before recording them permanently as snapshots in the repository history.

Command	Description & Notes
`git status`	Displays the current state of your working tree, showing tracked, modified, and untracked files.
`git add <file>`	Stages a specific file or set of files to prepare them for the next commit. Use `git add .` to stage all modifications.
`git commit -m "message"`	Records the staged snapshot permanently into the local repository history with a descriptive log message.
`git log --oneline`	Displays a condensed, single-line chronological list of recent project commits.

3. Branching and Navigation

Branches allow you to develop features, fix bugs, or experiment safely without disrupting the main production codebase.

Command	Description & Notes
`git branch`	Lists all local branches in your repository and highlights your currently active branch.
`git switch <branch>`	Switches your working tree to the specified target branch (a modern and safer alternative to `git checkout`).
`git switch -c <name>`	Creates a brand new branch and instantly switches to it in one single command.
`git merge <branch>`	Integrates the history and changes from the specified branch into your current active branch.

4. Remote Repositories & Collaboration

To work with group partners or back up your projects on GitHub, you need commands that synchronize local and remote histories.

Command	Description & Notes
`git remote add origin <url>`	Links your local repository to a remote server URL (such as GitHub).
`git fetch`	Downloads recent history updates and commits from the remote server without merging them into your working files.
`git pull`	Fetches updates from the remote repository and immediately merges them into your active local branch.
`git push -u origin <branch>`	Uploads your local branch commits to the remote repository and links the tracking upstream branch.

5. Undoing Changes & Troubleshooting

Mistakes happen during development. These commands help you inspect differences, unstage changes, or safely revert errors.

Command	Description & Notes
`git diff`	Shows exact line-by-line code differences that have been modified in your working tree but not yet staged.
`git restore <file>`	Discards uncommitted local changes made to a specific file, reverting it to the last committed state.
`git reset <file>`	Unstages a file while preserving your actual file modifications intact within the working directory.